Backend Integration Handoff & Advanced CMS Guide

Prepared for: Sultan (Senior Backend Engineer)
Project: IOCA (International Organization For Community Advancement) Website
Date: June 2026

Hello Sultan,

I hope you're doing well! The frontend architecture for the IOCA website has been fully completed and structurally prepared for your backend integration. We have meticulously organized the codebase to ensure that wiring up your backend APIs (Supabase or your preferred stack) will be a seamless, plug-and-play experience.

This document outlines the exact tasks required on the backend to make this website 100% dynamic and ready for the client.

1. Frontend Architecture Overview

To make your life easier, we have abstracted **all** data-fetching logic away from the UI components. You do not need to hunt through React components to connect your APIs.

- **The Service Layer:** All API calls are routed through a single file: `frontend/src/services/api.ts`.
- **The Types:** All TypeScript interfaces (schemas) are centralized in `frontend/src/types/index.ts`.
- **Environment Variables:** An `.env.example` file has been provided in the root directory.

Your Primary Goal: Swap out the mock data return statements inside `src/services/api.ts` with your actual asynchronous database queries.

---

2. Advanced Admin CMS Requirements

The client has requested the ability to fully manage the website without touching code. You will need to build the following upload/management flows in the Admin Dashboard:

A. Dynamic Gallery Management

The Gallery is now strictly categorized by the core programs.

- **Admin Flow:** Admin clicks "Add to Gallery" -> Uploads Image -> Selects Category (`education`, `health`, `youth`, or `community`) -> Enters Title -> Enters Description -> Clicks Save.
- **Database Schema (`gallery` table):**
    - `id` (uuid/serial)
    - `image_url` (string)
    - `titleEn` (string), `titleUr` (string)
    - `descEn` (string), `descUr` (string)
    - `category` (enum: 'education' | 'health' | 'youth' | 'community')
    - `created_at` (timestamp)
- **CRITICAL FETCH REQUIREMENT:** When fetching the gallery items for the frontend, you MUST order the query by `created_at DESC` (newest first). The client expects newly uploaded pictures to appear at the very top of the "All" tab and their respective category tabs.

B. Impact Stories Management

The client needs to be able to post new success stories dynamically.

- **Admin Flow:** Admin clicks "New Impact Story" -> Uploads Cover Image -> Enters Title -> Enters Excerpt -> Enters Full Story Content -> Enters Category -> Clicks Save.
- **Database Schema (`impact_stories` table):**
    - `image_url` (string)
    - `titleEn`, `titleUr`, `excerptEn`, `excerptUr`, `contentEn`, `contentUr` (strings)
    - `categoryEn`, `categoryUr` (strings)
    - `date` (string/date)

C. Global Assets & Hero Image Replacements

The client and their designer must be able to swap out **any** static image on the website (e.g., Home Page Hero Images, Project Cover Images, Section Backgrounds, Logos).

- **Admin Flow:** The dashboard should have a "Global Assets" or "Site Settings" section. The admin can see current images, click "Replace", upload a new one, and optionally update the description/title for that specific area.
- **Database Architecture (`global_assets` key-value table):**
    - `asset_key` (e.g., 'hero_community_bg', 'project_education_cover')
    - `image_url` (string)
    - `title_override` (optional string)
    - `desc_override` (optional string)
- **Frontend Hookup:** Update `src/services/api.ts` to pull these global assets so the frontend UI components use the dynamically uploaded hero/cover images instead of the local `/assets/` files.

---

3. Storage & Asset Management

To support the above CMS features, you must handle file storage.

Required Tasks:

1. Create a public storage bucket (e.g., `ioca-media` in Supabase Storage).
2. Ensure the bucket is publicly readable.
3. When admins upload images via the Admin Dashboard, the backend should return the public URL and save that `image_url` to the respective database table.

---

4. Admin Dashboard Authentication & Security

The frontend includes an Admin Dashboard route (`/admin`) designed for the client to manage content.

Required Tasks:

1. **Authentication:** Implement a secure login mechanism (e.g., Supabase Auth / JWT) for the `/admin` route.
2. **Row Level Security (RLS):** If using Supabase, ensure that RLS policies are strictly enforced:
    - **Public:** `SELECT` access only (Read-only for all tables).
    - **Authenticated Admins:** `INSERT`, `UPDATE`, `DELETE` privileges across all content tables.

---

5. Contact & Donation Handlers

There are two interactive user-facing forms that require backend processing logic:

1. **Contact Form (`sendContactEmail.ts`):** Implement an email service (e.g., Resend, SendGrid) to forward messages submitted on the Contact page to the NGO's official inbox.
2. **Donation Form (`saveDonation.ts`):** Store donation intents/records and integrate with any required payment gateways.

Summary Checklist for Handoff

- [ ] Database tables created (`projects`, `programs`, `gallery`, `impact_stories`, `global_assets`, etc.)
- [ ] Public storage bucket created for dynamic media uploads.
- [ ] `src/services/api.ts` updated to fetch from the live database.
- [ ] Admin Authentication implemented to secure the dashboard.
- [ ] CMS Upload flows built for Gallery, Impact Stories, and Global Asset swapping.
- [ ] Row Level Security (RLS) enabled.
- [ ] Contact form email forwarding implemented.

Sultan, thank you for taking the baton on this! The frontend is strictly typed and built to scale, so your APIs should map over perfectly. Please don't hesitate to reach out if you need any clarification on the frontend architecture or the type definitions.

Best of luck!